

Sección 2: Los Cimientos: Por qué la Arquitectura Importa

- Qué pasa cuando un proyecto crece sin criterio arquitectónico
- Setup del proyecto atlas-bank con Spring Initializr
- CRUD base funcional: controllers, services, repositories
- SOLID aplicado a Spring Boot: SRP, OCP, LSP, ISP, DIP
- Inyección de dependencias: constructor vs field injection
- @Qualifier y @Primary
- Package-by-feature vs package-by-layer: refactoring del proyecto

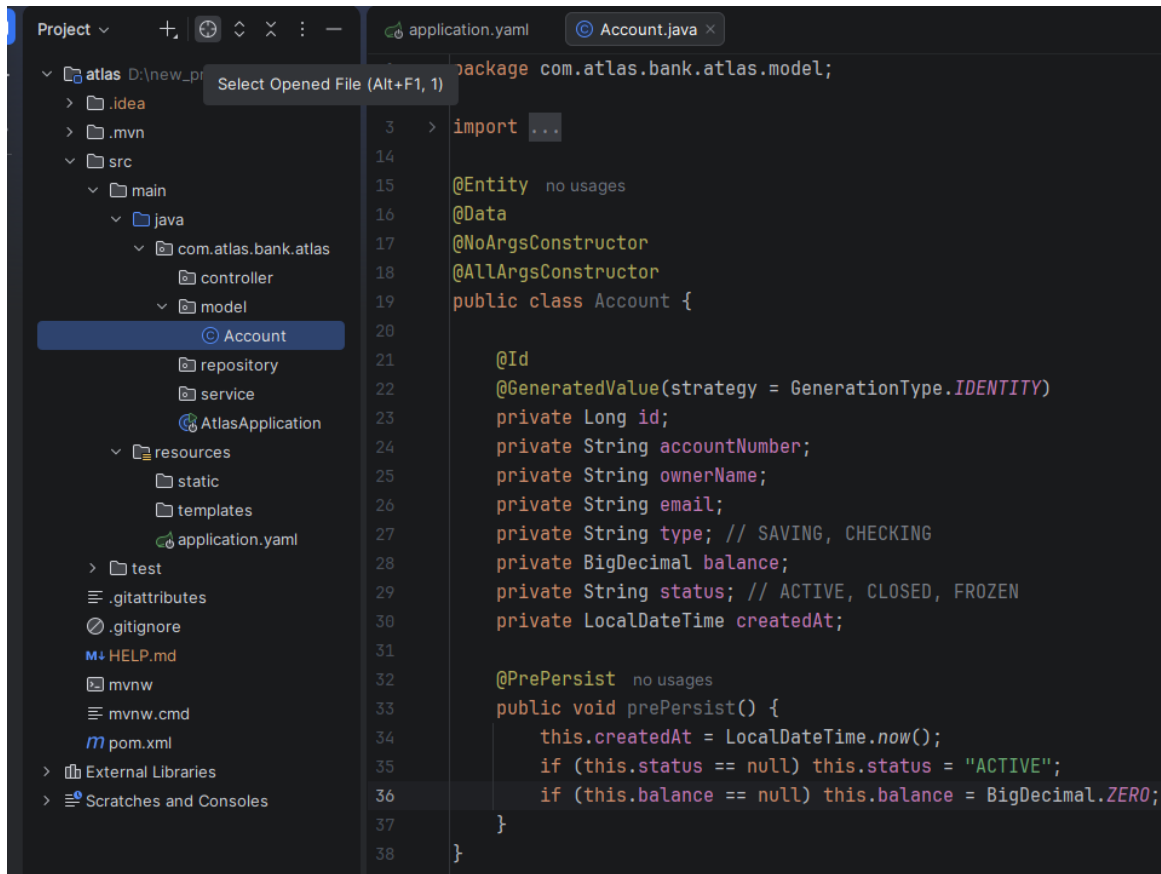
9. Setup del proyecto base - parte 1

The screenshot shows the Spring Initializr web interface for configuring a new project. The 'Project' section on the left includes options for 'Project' (Maven selected), 'Language' (Java selected), 'Spring Boot' (4.1.1 selected), and 'Project Metadata' (Group: com.atlas.bank, Artifact: atlas, Package name: com.atlas.bank.atlas, Packaging: Jar, Configuration: Yaml, Java: 21). The 'Dependencies' section on the right lists several dependencies: Spring Web (WEB), Lombok (DEVELOPER TOOLS), Validation (I/O), Spring Data JPA (SQL), and H2 Database (SQL). Each dependency has a brief description of its function.

The screenshot shows an IDE with the 'application.yaml' file open. The file contains the following configuration:

```
1  spring:
2    application:
3      name: atlas
4
5    datasource:
6      url: jdbc:h2:mem:atlasbank
7      driver-class-name: org.h2.Driver
8      username: sa
9      password:
10
11    jpa:
12      database-platform: org.hibernate.dialect.H2Dialect
13      hibernate:
14        ddl-auto: create-drop
15      show-sql: true
16
17    h2:
18      console:
19        enabled: true
20        path: /h2-console
21
22  server:
23    port: 8080
```

➔Primer modelo:



The screenshot shows the IntelliJ IDEA interface. On the left, the Project Explorer displays the project structure with the 'Account' entity selected under 'com.atlas.bank.atlas.model'. The main editor shows the code for 'Account.java'.

```
package com.atlas.bank.atlas.model;

import ...

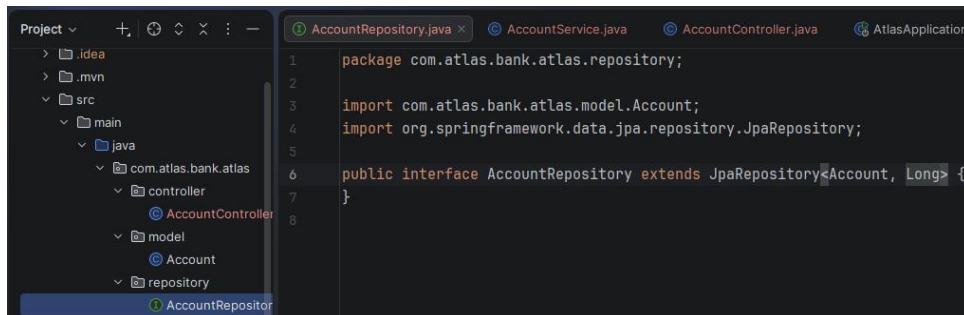
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String accountNumber;
    private String ownerName;
    private String email;
    private String type; // SAVING, CHECKING
    private BigDecimal balance;
    private String status; // ACTIVE, CLOSED, FROZEN
    private LocalDateTime createdAt;

    @PrePersist
    public void prePersist() {
        this.createdAt = LocalDateTime.now();
        if (this.status == null) this.status = "ACTIVE";
        if (this.balance == null) this.balance = BigDecimal.ZERO;
    }
}
```

10. Setup del proyecto base - parte 2

➔Repository



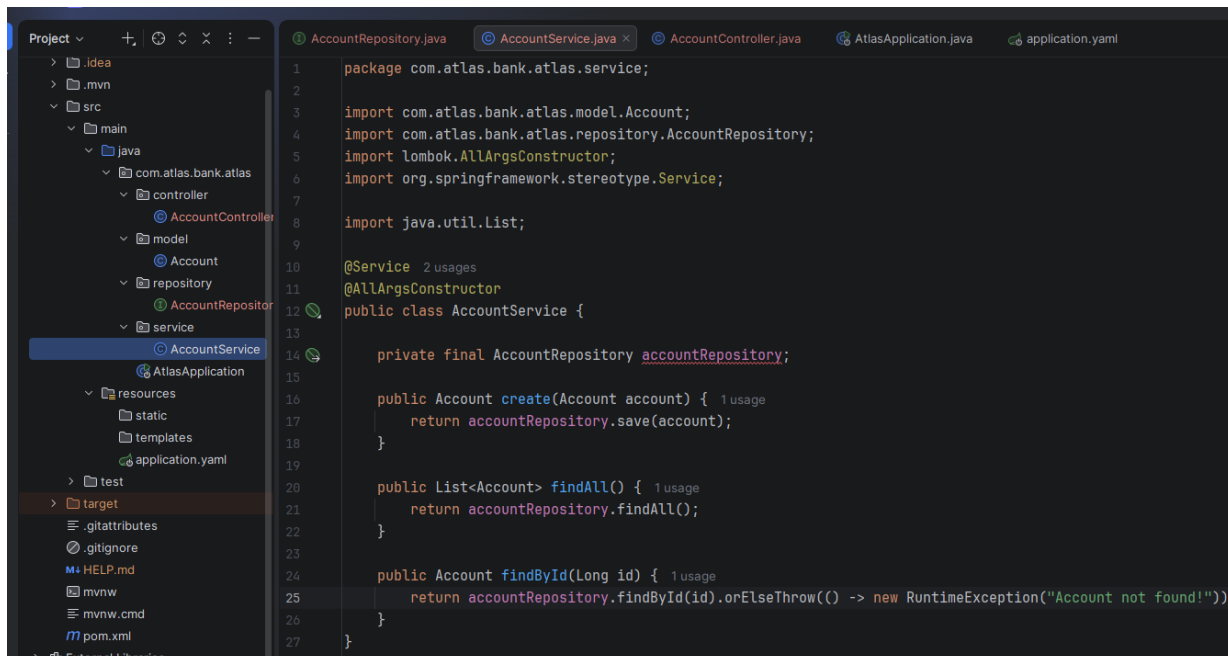
The screenshot shows the IntelliJ IDEA interface with the 'AccountRepository.java' file open. The Project Explorer on the left shows the 'AccountRepository' interface selected under 'com.atlas.bank.atlas.repository'.

```
package com.atlas.bank.atlas.repository;

import com.atlas.bank.atlas.model.Account;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AccountRepository extends JpaRepository<Account, Long> {
}
```

➔Service



The screenshot shows the IntelliJ IDEA interface with the 'AccountService.java' file open. The Project Explorer on the left shows the 'AccountService' class selected under 'com.atlas.bank.atlas.service'.

```
package com.atlas.bank.atlas.service;

import com.atlas.bank.atlas.model.Account;
import com.atlas.bank.atlas.repository.AccountRepository;
import lombok.AllArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@AllArgsConstructor
public class AccountService {

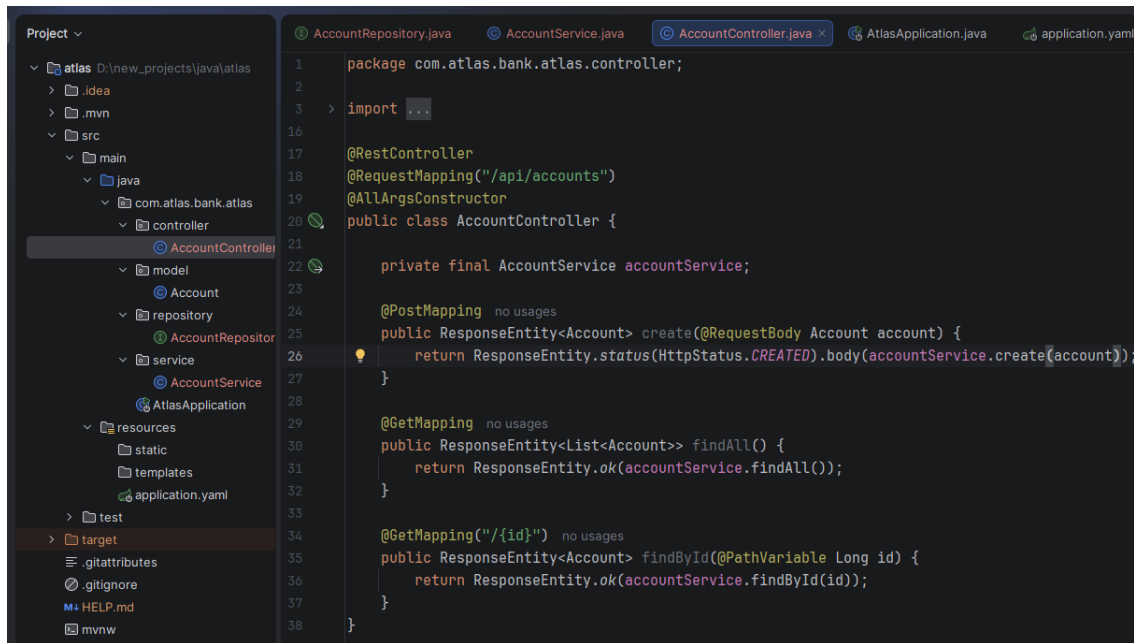
    private final AccountRepository accountRepository;

    public Account create(Account account) {
        return accountRepository.save(account);
    }

    public List<Account> findAll() {
        return accountRepository.findAll();
    }

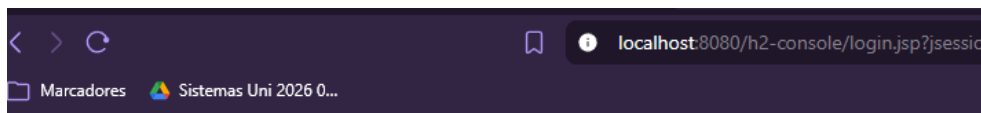
    public Account findById(Long id) {
        return accountRepository.findById(id).orElseThrow(() -> new RuntimeException("Account not found!"));
    }
}
```

→ Controller

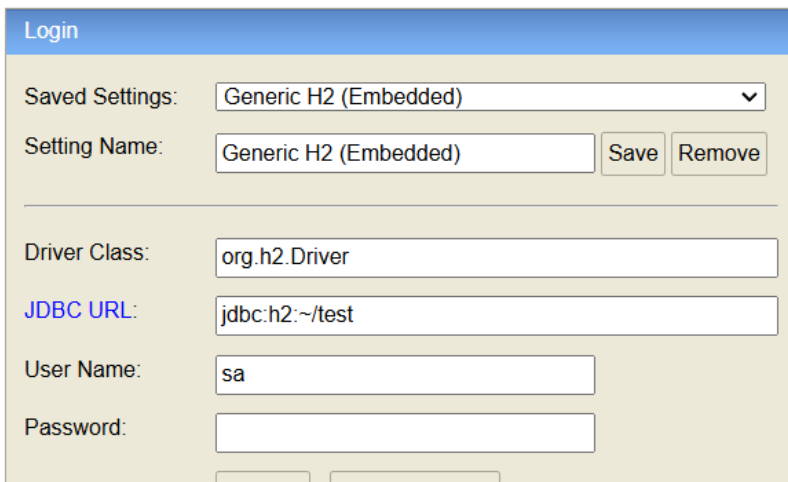


```
1 package com.atlas.bank.atlas.controller;
2
3 import ...
4
5 @RestController
6 @RequestMapping("/api/accounts")
7 @AllArgsConstructor
8 public class AccountController {
9
10     private final AccountService accountService;
11
12     @PostMapping
13     public ResponseEntity<Account> create(@RequestBody Account account) {
14         return ResponseEntity.status(HttpStatus.CREATED).body(accountService.create(account));
15     }
16
17     @GetMapping
18     public ResponseEntity<List<Account>> findAll() {
19         return ResponseEntity.ok(accountService.findAll());
20     }
21
22     @GetMapping("/{id}")
23     public ResponseEntity<Account> findById(@PathVariable Long id) {
24         return ResponseEntity.ok(accountService.findById(id));
25     }
26 }
```

→ Entrar a h2: <http://localhost:8080/h2-console/>



English Preferences Tools Help



Login

Saved Settings: Generic H2 (Embedded) ▼

Setting Name: Generic H2 (Embedded) Save Remove

Driver Class: org.h2.Driver

JDBC URL: jdbc:h2:~/test

User Name: sa

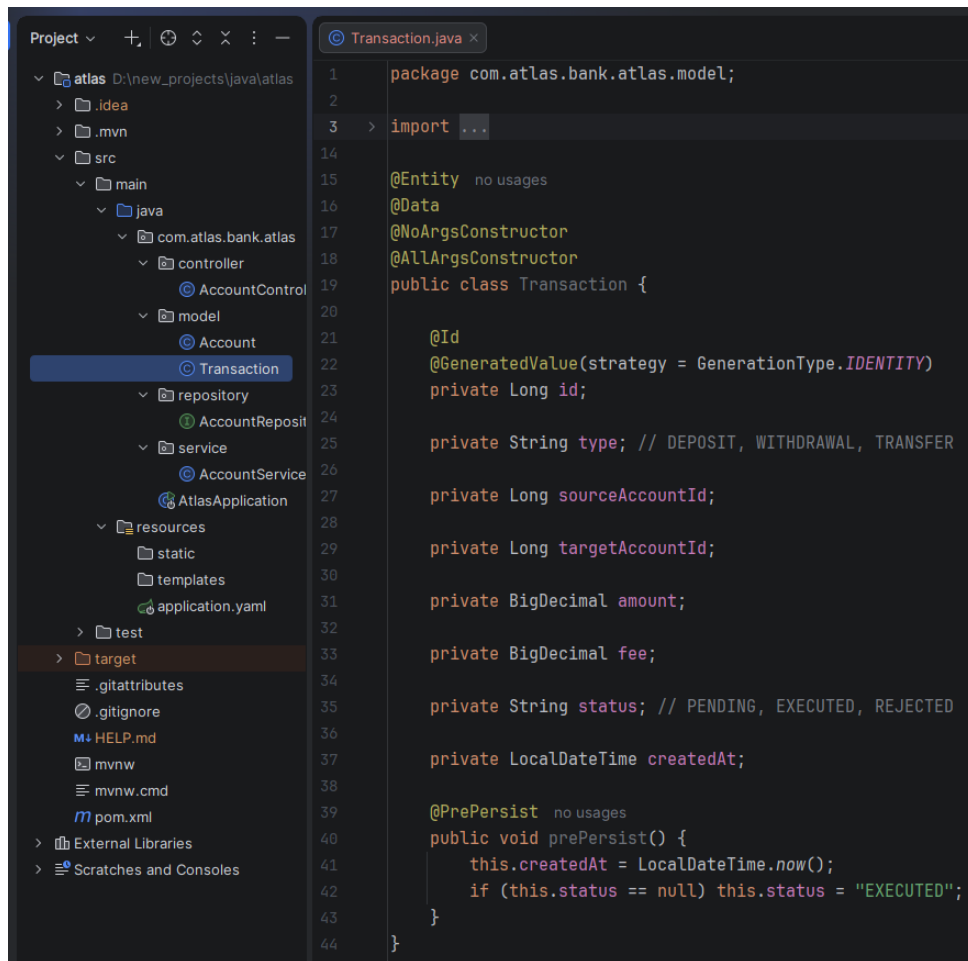
Password:

url: jdbc:h2:mem:atlasbank

11. Conectando las capas del proyecto 1 y 2

<https://gist.github.com/gchaldu/ca84a10448026c4ca3550a938a3e5028>

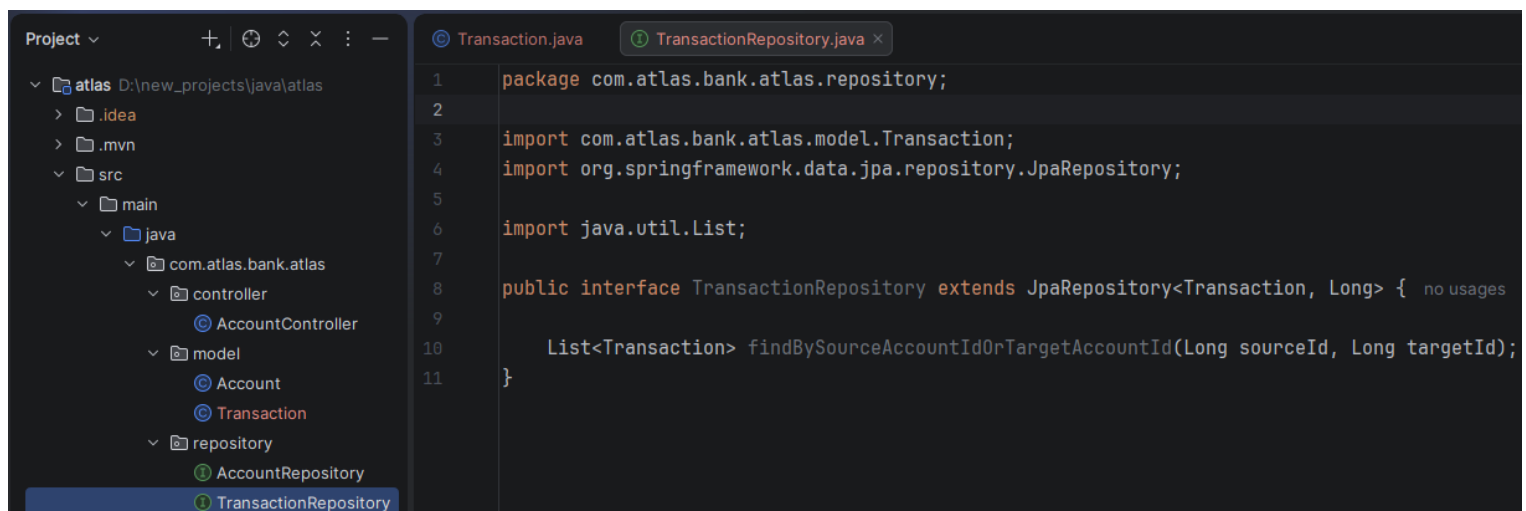
→ Modelo:



The screenshot shows an IDE with a project named 'atlas' in the left sidebar. The project structure includes 'src/main/java/com.atlas.bank.atlas'. The 'Transaction.java' file is open in the editor. The code defines a JPA entity for transactions.

```
1 package com.atlas.bank.atlas.model;
2
3 > import ...
4
14
15 @Entity no usages
16 @Data
17 @NoArgsConstructor
18 @AllArgsConstructor
19 public class Transaction {
20
21     @Id
22     @GeneratedValue(strategy = GenerationType.IDENTITY)
23     private Long id;
24
25     private String type; // DEPOSIT, WITHDRAWAL, TRANSFER
26
27     private Long sourceAccountId;
28
29     private Long targetAccountId;
30
31     private BigDecimal amount;
32
33     private BigDecimal fee;
34
35     private String status; // PENDING, EXECUTED, REJECTED
36
37     private LocalDateTime createdAt;
38
39     @PrePersist no usages
40     public void prePersist() {
41         this.createdAt = LocalDateTime.now();
42         if (this.status == null) this.status = "EXECUTED";
43     }
44 }
```

→ Repo:



The screenshot shows the same IDE with the 'TransactionRepository.java' file open in the editor. The code defines a JPA repository interface for transactions.

```
1 package com.atlas.bank.atlas.repository;
2
3 import com.atlas.bank.atlas.model.Transaction;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.util.List;
7
8 public interface TransactionRepository extends JpaRepository<Transaction, Long> { no usages
9
10     List<Transaction> findBySourceAccountIdOrTargetAccountId(Long sourceId, Long targetId);
11 }
```

➔Modificando service:

```
package com.atlas.bank.atlas.service;
```

```
import com.atlas.bank.atlas.model.Account;
import com.atlas.bank.atlas.model.Transaction;
import com.atlas.bank.atlas.repository.AccountRepository;
import com.atlas.bank.atlas.repository.TransactionRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
import java.math.BigDecimal;
import java.util.List;
```

```
@Service
```

```
@RequiredArgsConstructor
```

```
public class AccountService {
```

```
    private final AccountRepository accountRepository;
    private final TransactionRepository transactionRepository;
```

```
    public Account create(Account account) {
        return accountRepository.save(account);
    }
```

```
    public List<Account> findAll() {
        return accountRepository.findAll();
    }
```

```
    public Account findById(Long id) {
        return accountRepository.findById(id).orElseThrow(() -> new RuntimeException("Account not found!"));
    }
```

```
@Transactional
```

```
    public Transaction transfer(Long fromId, Long toId, BigDecimal amount) {
        // Buscar cuentas
        Account from = accountRepository.findById(fromId)
            .orElseThrow(() -> new RuntimeException("Cuenta origen no encontrada"));
        Account to = accountRepository.findById(toId)
            .orElseThrow(() -> new RuntimeException("Cuenta destino no encontrada"));
```

```
        // Validar que la cuenta esté activa
        if (!"ACTIVE".equals(from.getStatus())) {
            throw new RuntimeException("La cuenta origen no está activa");
        }
        if (!"ACTIVE".equals(to.getStatus())) {
            throw new RuntimeException("La cuenta destino no está activa");
        }
    }
}
```

```

}

// Validar fondos
if (from.getBalance().compareTo(amount) < 0) {
    throw new RuntimeException("Fondos insuficientes");
}

// Calcular comisión — hardcodeada
BigDecimal fee;
if ("SAVINGS".equals(from.getType())) {
    fee = amount.multiply(new BigDecimal("0.01"));
} else if ("CHECKING".equals(from.getType())) {
    fee = amount.multiply(new BigDecimal("0.015"));
} else {
    fee = BigDecimal.ZERO;
}

// Actualizar saldos
from.setBalance(from.getBalance().subtract(amount).subtract(fee));
to.setBalance(to.getBalance().add(amount));
accountRepository.save(from);
accountRepository.save(to);

// Crear transacción
Transaction transaction = new Transaction();
transaction.setType("TRANSFER");
transaction.setSourceAccountId(fromId);
transaction.setTargetAccountId(toId);
transaction.setAmount(amount);
transaction.setFee(fee);
transaction.setStatus("EXECUTED");

return transactionRepository.save(transaction);
}

public List<Transaction> getTransactions(Long accountId) {
    return transactionRepository
        .findBySourceAccountIdOrTargetAccountId(accountId, accountId);
}
}

```

➔Controlador:

Project

atlas

D:\new_projects\java\atlas

idea

.idea

.mvn

src

main

java

com.atlas.bank.atlas

controller

AccountController

model

Account

Transaction

repository

AccountRepository

TransactionRepository

service

AccountService

AtlasApplication

resources

static

templates

application.yaml

test

target

.gitattributes

.gitignore

HELP.md

mvnw

mvnw.cmd

pom.xml

External Libraries

Scratches and Consoles

AccountController.java

Account.java

```
19
20 @RestController @ydochoadev *
21 @RequestMapping("/api/accounts")
22 @AllArgsConstructor
23 public class AccountController {
24
25     private final AccountService accountService;
26
27     @PostMapping no usages @ydochoadev
28     public ResponseEntity<Account> create(@RequestBody Account account) {
29         return ResponseEntity.status(HttpStatus.CREATED).body(accountService.create(account));
30     }
31
32     @GetMapping no usages @ydochoadev
33     public ResponseEntity<List<Account>> findAll() { return ResponseEntity.ok(accountService.findAll()); }
34
35     @GetMapping("/{id}") no usages @ydochoadev
36     public ResponseEntity<Account> findById(@PathVariable Long id) {
37         return ResponseEntity.ok(accountService.findById(id));
38     }
39
40     @PostMapping("/transfer") no usages new *
41     public ResponseEntity<Transaction> transfer(@RequestParam Long fromId,
42                                                 @RequestParam Long toId,
43                                                 @RequestParam BigDecimal amount) {
44         return ResponseEntity.ok(accountService.transfer(fromId, toId, amount));
45     }
46
47     @GetMapping("/{id}/transactions") no usages new *
48     public ResponseEntity<List<Transaction>> getTransactions(@PathVariable Long id) {
49         return ResponseEntity.ok(accountService.getTransactions(id));
50     }
51
52 }
53
54 }
```

13. Principios SOLID

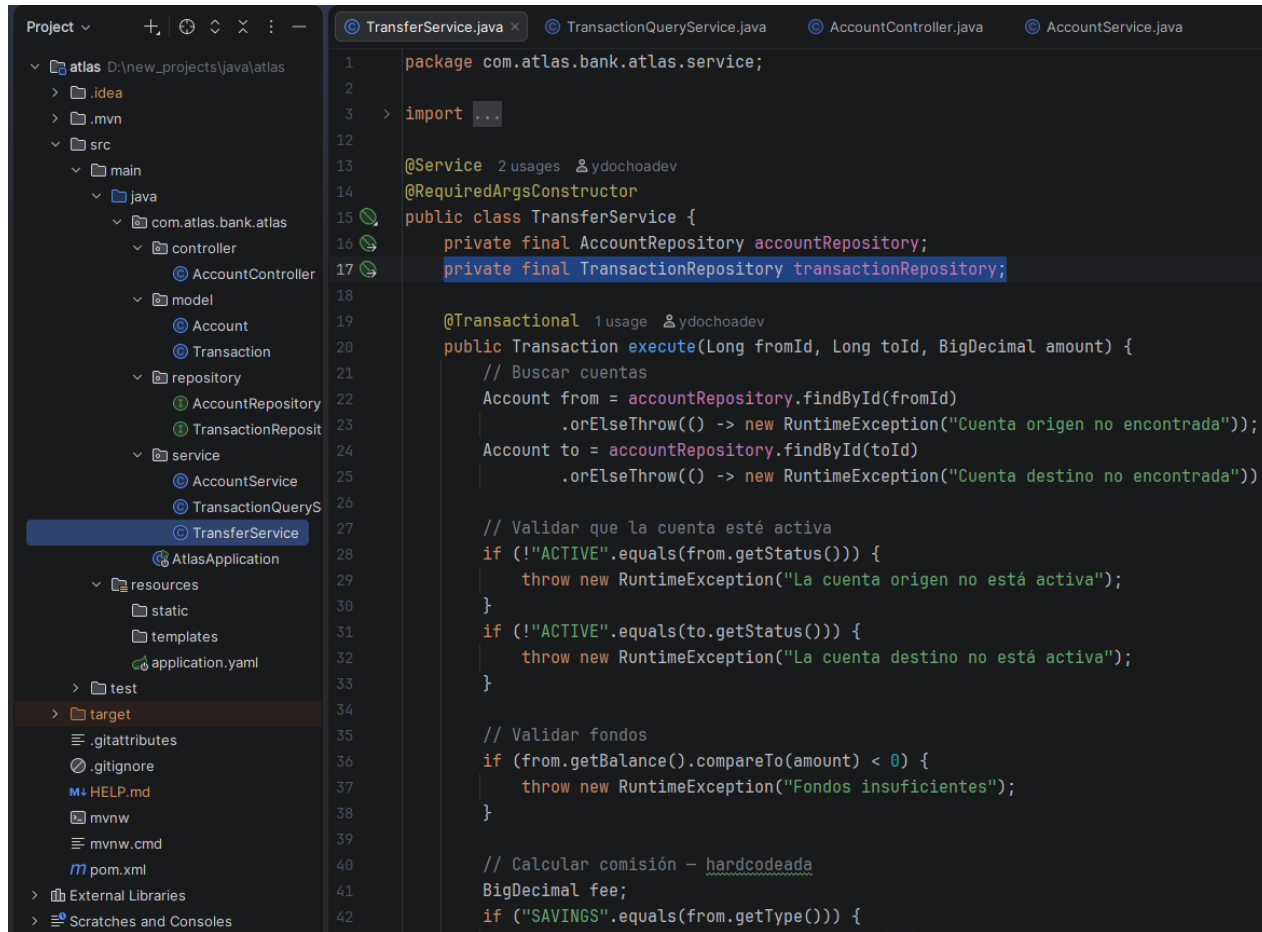
- **S** — Single Responsibility
- **O** — Open/Closed
- **L** — Liskov Substitution
- **I** — Interface Segregation
- **D** — Dependency Inversion

Principio	Problema en atlas-bank	Qué vamos a hacer
S — SRP	AccountService hace 5 cosas	Partir en servicios cohesivos
O — OCP	if/else para comisiones	Strategy que se extiende sin modificar
L — LSP	Herencia que rompe contratos	Subtipos que respetan el contrato
I — ISP	Interfaces gordas	Contratos específicos por cliente
D — DIP	Service conoce la implementación	Depender de abstracciones

14. Single Responsibility: el servicio que hace todo

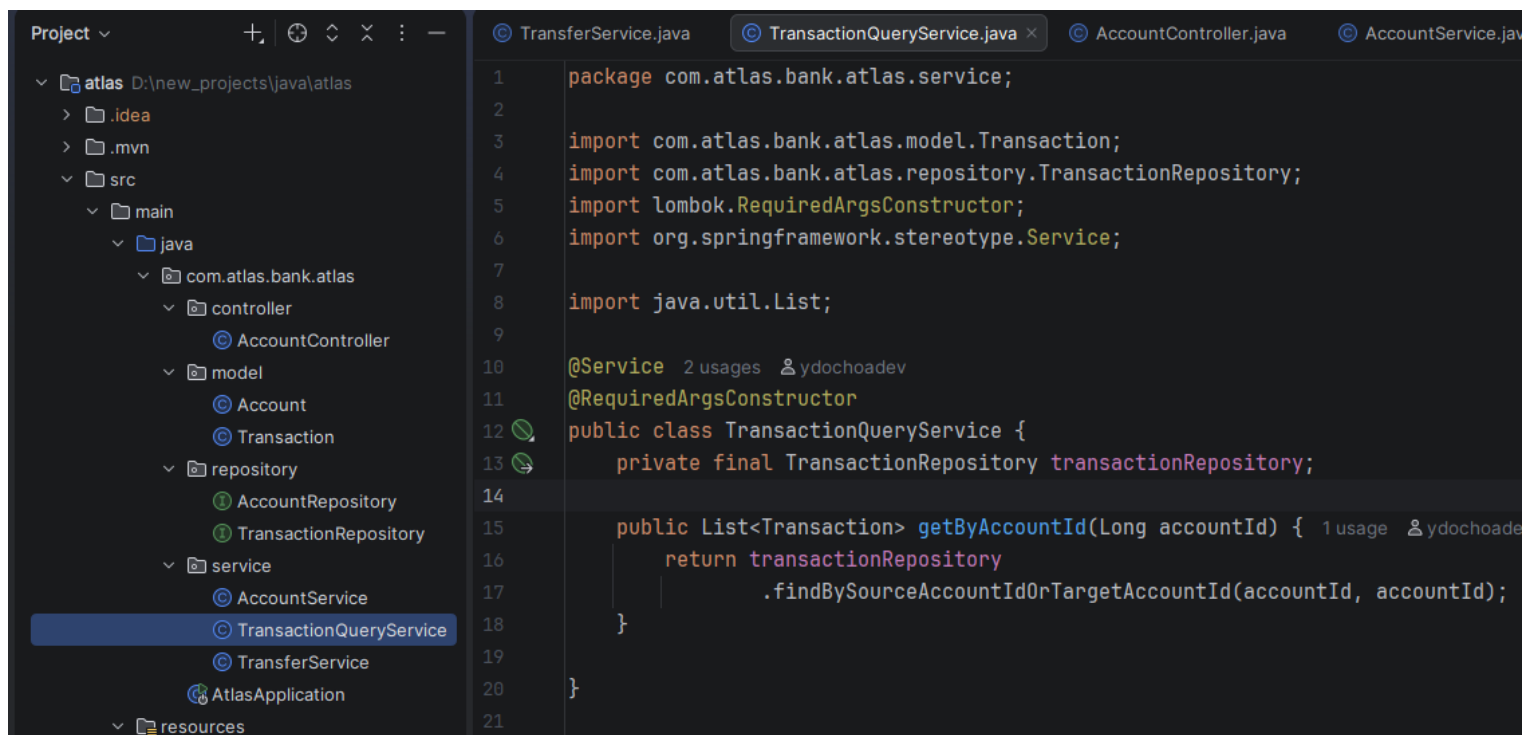
→ Una clase debe tener una razón para cambiar: Si hay varias razones para cambiar una clase, no cumple con este principio.

→ Creando clase solo para transferencia:



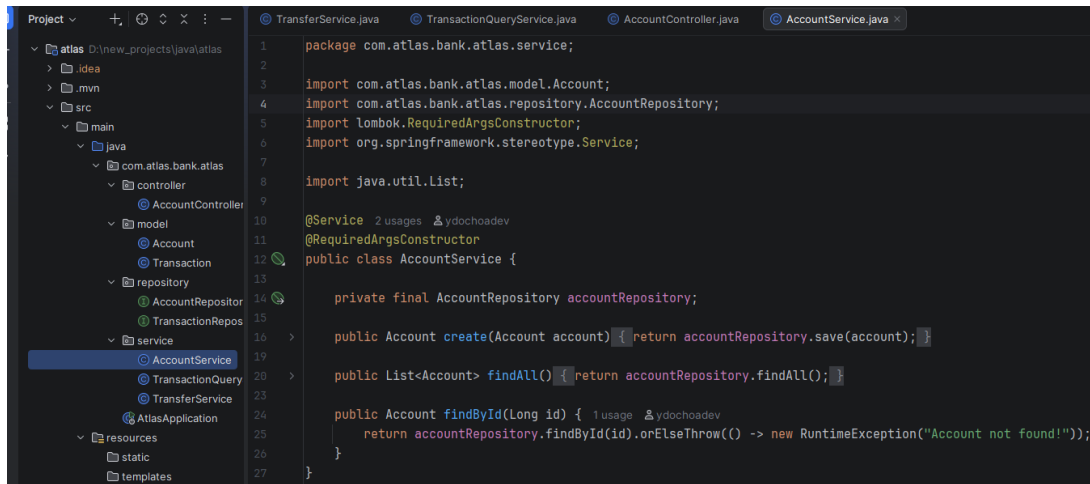
```
1 package com.atlas.bank.atlas.service;
2
3 import ...
4
5 @Service 2 usages ydochoadev
6 @RequiredArgsConstructor
7 public class TransferService {
8     private final AccountRepository accountRepository;
9     private final TransactionRepository transactionRepository;
10
11     @Transactional 1 usage ydochoadev
12     public Transaction execute(Long fromId, Long toId, BigDecimal amount) {
13         // Buscar cuentas
14         Account from = accountRepository.findById(fromId)
15             .orElseThrow(() -> new RuntimeException("Cuenta origen no encontrada"));
16         Account to = accountRepository.findById(toId)
17             .orElseThrow(() -> new RuntimeException("Cuenta destino no encontrada"));
18
19         // Validar que la cuenta esté activa
20         if (!"ACTIVE".equals(from.getStatus())) {
21             throw new RuntimeException("La cuenta origen no está activa");
22         }
23         if (!"ACTIVE".equals(to.getStatus())) {
24             throw new RuntimeException("La cuenta destino no está activa");
25         }
26
27         // Validar fondos
28         if (from.getBalance().compareTo(amount) < 0) {
29             throw new RuntimeException("Fondos insuficientes");
30         }
31
32         // Calcular comisión - hardcodeada
33         BigDecimal fee;
34         if ("SAVINGS".equals(from.getType())) {
35             fee = new BigDecimal("0.05");
36         } else {
37             fee = new BigDecimal("0.02");
38         }
39
40         Transaction transaction = new Transaction(from, to, amount, fee);
41         transactionRepository.save(transaction);
42     }
43 }
```

→ Clase para consulta de transacción:



```
1 package com.atlas.bank.atlas.service;
2
3 import com.atlas.bank.atlas.model.Transaction;
4 import com.atlas.bank.atlas.repository.TransactionRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7
8 import java.util.List;
9
10 @Service 2 usages ydochoadev
11 @RequiredArgsConstructor
12 public class TransactionQueryService {
13     private final TransactionRepository transactionRepository;
14
15     public List<Transaction> getByAccountId(Long accountId) { 1 usage ydochoadev
16         return transactionRepository
17             .findBySourceAccountIdOrTargetAccountId(accountId, accountId);
18     }
19
20 }
21 }
```


➔ AccountService quedaría:

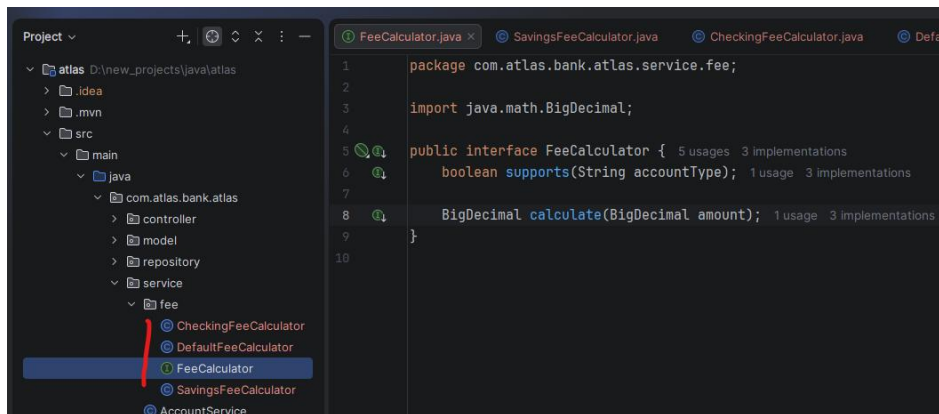


```
1 package com.atlas.bank.atlas.service;
2
3 import com.atlas.bank.atlas.model.Account;
4 import com.atlas.bank.atlas.repository.AccountRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7
8 import java.util.List;
9
10 @Service
11 @RequiredArgsConstructor
12 public class AccountService {
13
14     private final AccountRepository accountRepository;
15
16     public Account create(Account account) { return accountRepository.save(account); }
17
18     public List<Account> findAll() { return accountRepository.findAll(); }
19
20     public Account findById(Long id) {
21         return accountRepository.findById(id).orElseThrow(() -> new RuntimeException("Account not found!"));
22     }
23 }
24
25
26
27
```

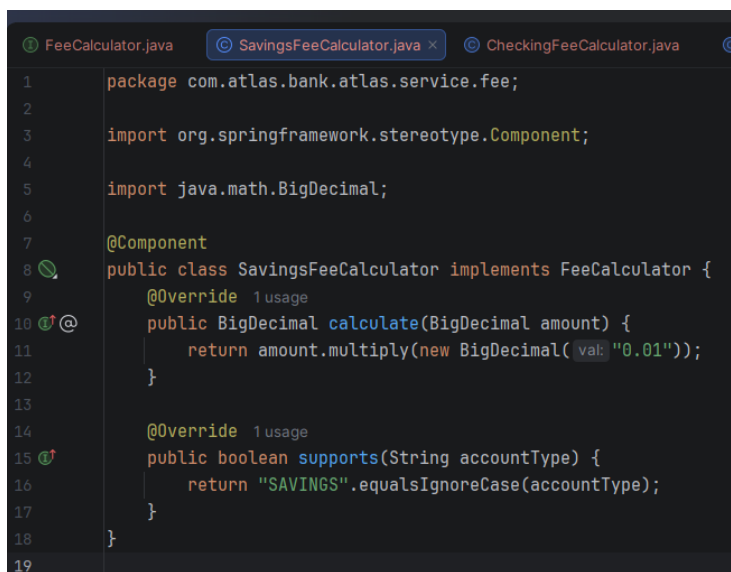
➔ Cambiar el controlador por las clases creadas.

15. Open Closed: extender sin romper

➔ Se crea una interfaz con varias implementaciones para calcular la comisión:



```
1 package com.atlas.bank.atlas.service.fee;
2
3 import java.math.BigDecimal;
4
5 public interface FeeCalculator {
6     boolean supports(String accountType);
7
8     BigDecimal calculate(BigDecimal amount);
9 }
10
```



```
1 package com.atlas.bank.atlas.service.fee;
2
3 import org.springframework.stereotype.Component;
4
5 import java.math.BigDecimal;
6
7 @Component
8 public class SavingsFeeCalculator implements FeeCalculator {
9     @Override
10     public BigDecimal calculate(BigDecimal amount) {
11         return amount.multiply(new BigDecimal("0.01"));
12     }
13
14     @Override
15     public boolean supports(String accountType) {
16         return "SAVINGS".equalsIgnoreCase(accountType);
17     }
18 }
19
```

```
FeeCalculator.java SavingsFeeCalculator.java CheckingFeeCalculator.java x
1 package com.atlas.bank.atlas.service.fee;
2
3 import org.springframework.stereotype.Component;
4
5 import java.math.BigDecimal;
6
7 @Component
8 public class CheckingFeeCalculator implements FeeCalculator {
9     @Override 1 usage
10     public BigDecimal calculate(BigDecimal amount) {
11         return amount.multiply(new BigDecimal("0.015"));
12     }
13
14     @Override 1 usage
15     public boolean supports(String accountType) {
16         return "CHECKING".equalsIgnoreCase(accountType);
17     }
18 }
```

```
FeeCalculator.java SavingsFeeCalculator.java CheckingFeeCalculator.java DefaultFeeCalculator.java x
1 package com.atlas.bank.atlas.service.fee;
2
3 import org.springframework.stereotype.Component;
4
5 import java.math.BigDecimal;
6
7 @Component
8 public class DefaultFeeCalculator implements FeeCalculator {
9     @Override 1 usage
10     public BigDecimal calculate(BigDecimal amount) { return BigDecimal.ZERO; }
11
12
13
14     @Override 1 usage
15     public boolean supports(String accountType) { return true; }
16
17 }
18
19
```

****** private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones cuando se inyecta la interfaz

```
FeeCalculator.java SavingsFeeCalculator.java CheckingFeeCalculator.java DefaultFeeCalculator.java TransferService.java x
1 package com.atlas.bank.atlas.service;
2
3 > import ...
14
15 @Service 2 usages ydochoadev *
16 @RequiredArgsConstructor
17 public class TransferService {
18     private final AccountRepository accountRepository;
19     private final TransactionRepository transactionRepository;
20     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones ✓
21
22 @Transactional 1 usage ydochoadev *
23 public Transaction execute(Long fromId, Long toId, BigDecimal amount) {
24     // Buscar cuentas
25     Account from = accountRepository.findById(fromId)
26         .orElseThrow(() -> new RuntimeException("Cuenta origen no encontrada"));
27     Account to = accountRepository.findById(toId)
28         .orElseThrow(() -> new RuntimeException("Cuenta destino no encontrada"));
29
30     // Validar que la cuenta esté activa
31     if (!"ACTIVE".equals(from.getStatus())) {
32         throw new RuntimeException("La cuenta origen no está activa");
33     }
34     if (!"ACTIVE".equals(to.getStatus())) {
35         throw new RuntimeException("La cuenta destino no está activa");
36     }
37
38     // Validar fondos
39     if (from.getBalance().compareTo(amount) < 0) {
40         throw new RuntimeException("Fondos insuficientes");
41     }
42
43     // Calcular comisión
44     BigDecimal fee = this.feeCalculators.stream()
45         .filter( FeeCalculator fc -> fc.supports(from.getType()))
46         .findFirst()
47         .orElseThrow(() -> new RuntimeException("No hay calculador para el tipo " + from.getType()))
48         .calculate(amount);
49
50 }
```

16. Liskov: herencia que no miente

→ Si S es subtipo de T, cualquier instancia de T debería poder reemplazarse por S sin alterar el comportamiento: En la herencia, el hijo puede sustituir al padre sin romper nada.

** Si el hijo no puede hacer lo que hace el padre, no debería heredar de él.

→ ¿Cuándo pensar en Liskov?

- Cuando usás herencia entre clases (**extends**)
- Cuando sobrescribís métodos (**@Override**)
- Cuando un subtipo lanza excepciones inesperadas
- Cuando un subtipo ignora parámetros o no produce efecto
- Pregunta clave: ¿Puedo usar cualquier hijo en el lugar del padre sin que nada se rompa?

java

```
interface CuentaBancaria {  
    void depositar(double monto);  
}  
  
interface CuentaRetirable extends CuentaBancaria {  
    void retirar(double monto);  
}  
  
class CuentaCorriente implements CuentaRetirable {  
    public void depositar(double monto) {  
        System.out.println("Depósito de: " + monto);  
    }  
  
    public void retirar(double monto) {  
        System.out.println("Retiro de: " + monto);  
    }  
}  
  
class CuentaInversion implements CuentaBancaria {  
    public void depositar(double monto) {  
        System.out.println("Depósito en inversión de: " + monto);  
    }  
}
```

Usa el código con precaución.



Ahora, cualquier objeto que implemente `CuentaRetirable` puede sustituir a otro sin romper el código que realiza retiros.  LearnYard +1

17. Interface segregation: interfaces que no estorban

→ ISP: Ningún cliente debería depender de métodos que no utiliza: Si nosotros tenemos una interfaz con 10 métodos y una clase que sólo necesita dos; esa clase, si nosotros implementamos esa interfaz, está obligada a conocer los otros ocho que no le importan. Eso va a generar un acoplamiento innecesario.

Se debe partir la interfaz en contratos más pequeños y específicos.

→ ¿Cuándo pensar en ISP?

- Cuando una interfaz tiene más de 5-6 métodos
- Cuando distintos clientes usan subconjuntos diferentes
- Cuando un cambio afecta clases que no usan ese método
- Cuando definís contratos entre capas (**puertos en Hexagonal**)
- Pregunta clave: **¿esta clase necesita TODOS estos métodos?**

18. Dependency inversion: depender de abstracciones

→ DIP: Depender de abstracciones, no de implementaciones concretas. El código no debería conocer de cómo se hacen las cosas. El QUÉ es la interfaz; el CÓMO es la implementación concreta.

→ El controlador depende de clases concretas y esto no debería de pasar: se está violando el principio de inversión de dependencias:

```
AccountController.java x
1 package com.atlas.bank.atlas.controller;
2
3 > import ...
21
22 @RestController @ydochoadev
23 @RequestMapping("/api/accounts")
24 @AllArgsConstructor
25 public class AccountController {
26
27     private final AccountService accountService;
28     private final TransferService transferService;
29     private final TransactionQueryService transactionQueryService;
```

→ Crear interface para las cuentas:

```
Project v
+ - - - - -
+ AccountController.java
+ IAccountService.java x

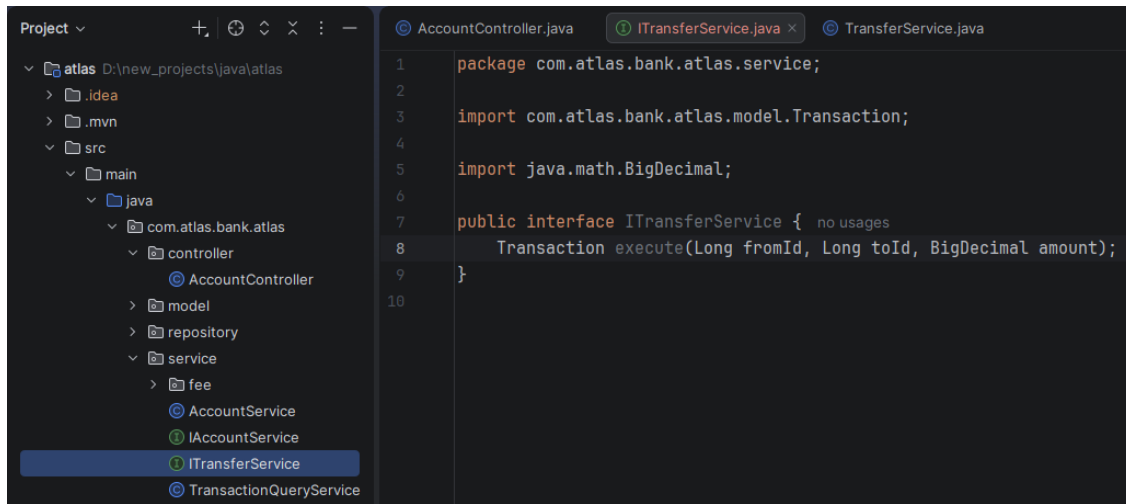
Select Opened File (Alt+F1, 1)
package com.atlas.bank.atlas.service;
2
3 import com.atlas.bank.atlas.model.Account;
4
5 import java.util.List;
6
7 public interface IAccountService { no usages
8     Account createAccount(Account account);
9
10    List<Account> findAll(); no usages
11
12    Account findById(Long id);
13 }
14
```

Modificar el AccountService para que implemente de esta interfaz:

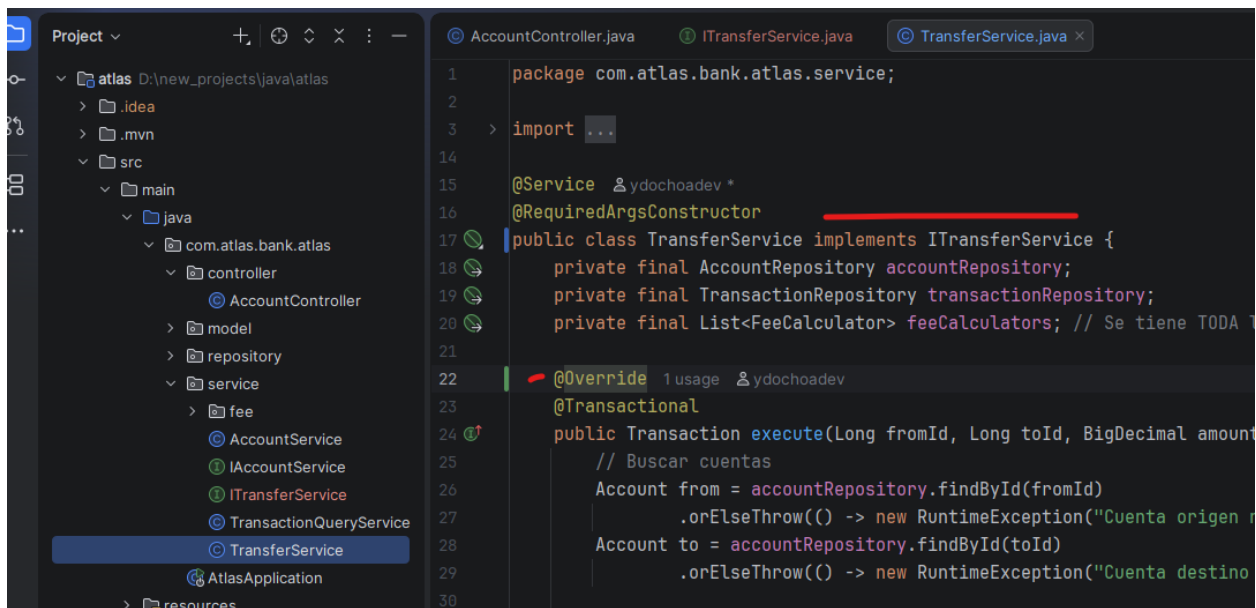
```
Project v
+ AccountController.java
+ IAccountService.java
+ AccountService.java x

package com.atlas.bank.atlas.service;
2
3 > import ...
9
10 @Service @ydochoadev *
11 @RequiredArgsConstructor
12 public class AccountService implements IAccountService {
13
14     private final AccountRepository accountRepository;
15
16     @Override 1 usage @ydochoadev
17     public Account create(Account account) { return accountRepository.save(account); }
20
21     @Override 1 usage @ydochoadev
22     public List<Account> findAll() { return accountRepository.findAll(); }
25
26     @Override 1 usage @ydochoadev
27     public Account findById(Long id) {
28         return accountRepository.findById(id).orElseThrow(() -> new RuntimeException("Account not found!"));
29     }
30 }
```

→ Lo mismo para las transferencias:

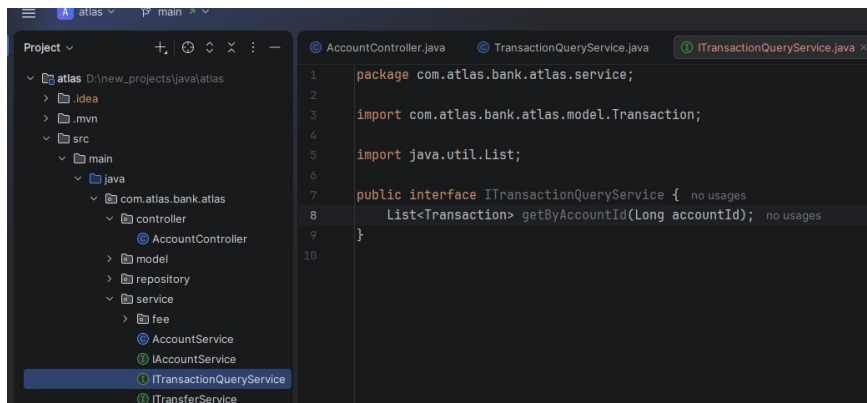


```
1 package com.atlas.bank.atlas.service;
2
3 import com.atlas.bank.atlas.model.Transaction;
4
5 import java.math.BigDecimal;
6
7 public interface ITransferService { no usages
8     Transaction execute(Long fromId, Long toId, BigDecimal amount);
9 }
10
```

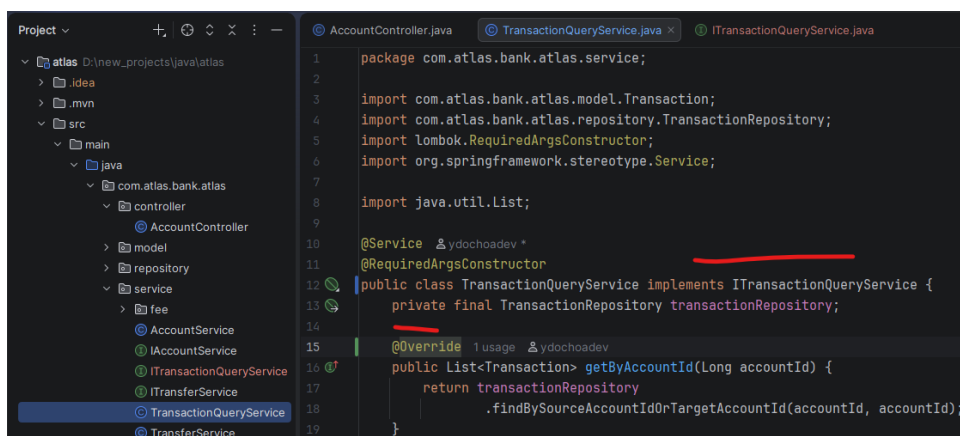


```
1 package com.atlas.bank.atlas.service;
2
3 import ...
4
5 @Service @ydochoadev *
6 @RequiredArgsConstructor
7 public class TransferService implements ITransferService {
8     private final AccountRepository accountRepository;
9     private final TransactionRepository transactionRepository;
10    private final List<FeeCalculator> feeCalculators; // Se tiene TODA T
11
12    @Override 1 usage @ydochoadev
13    @Transactional
14    public Transaction execute(Long fromId, Long toId, BigDecimal amount) {
15        // Buscar cuentas
16        Account from = accountRepository.findById(fromId)
17            .orElseThrow(() -> new RuntimeException("Cuenta origen no encontrada"));
18        Account to = accountRepository.findById(toId)
19            .orElseThrow(() -> new RuntimeException("Cuenta destino no encontrada"));
20    }
21 }
```

→ Otra para las querys de las transacciones:



```
1 package com.atlas.bank.atlas.service;
2
3 import com.atlas.bank.atlas.model.Transaction;
4
5 import java.util.List;
6
7 public interface ITransactionQueryService { no usages
8     List<Transaction> getByAccountId(Long accountId); no usages
9 }
10
```



```
1 package com.atlas.bank.atlas.service;
2
3 import com.atlas.bank.atlas.model.Transaction;
4 import com.atlas.bank.atlas.repository.TransactionRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7
8 import java.util.List;
9
10 @Service @ydochoadev *
11 @RequiredArgsConstructor
12 public class TransactionQueryService implements ITransactionQueryService {
13     private final TransactionRepository transactionRepository;
14
15     @Override 1 usage @ydochoadev
16     public List<Transaction> getByAccountId(Long accountId) {
17         return transactionRepository
18             .findBySourceAccountIdOrTargetAccountId(accountId, accountId);
19     }
20 }
```

→ Controlador:

```
AccountController.java x
1 package com.atlas.bank.atlas.controller;
2
3 import ...
21
22 @RestController @ydochoadev *
23 @RequestMapping("/api/accounts")
24 @AllArgsConstructor
25 public class AccountController {
26
27     private final IAccountService accountService;
28     private final ITransferService transferService;
29     private final ITransactionQueryService transactionQueryService;
30 }
```

19. Inyección de dependencia: Qué es y cómo funciona

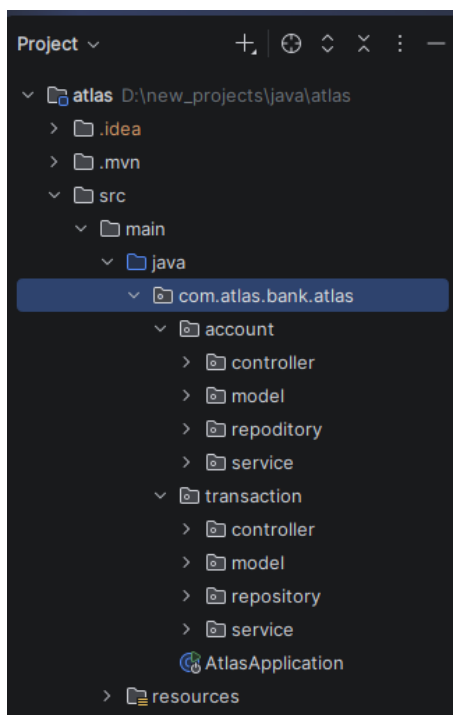
→ Que Spring te dé una instancia lista para usar, sin que uno haga un “new”.

- Spring crea **UNA** instancia de cada bean al arrancar
- Esa instancia se reutiliza en toda la aplicación
- 3 controllers necesitan AccountService → los 3 reciben **el mismo objeto**
- No hay múltiples copias en memoria
- Ese comportamiento se llama **Singleton** (scope por defecto)

20. Package-by-layer vs Package-by-feature

→ layer, paquetes por lo que hace, controlador, service, model, etc.

→ feature, ordenar los paquetes por features, transacciones, cuentas, etc.



→ Refactorizar.